

David Sandoval

Prof. Gary Thai

CMSC 204

February 7, 2026

During the 1st project of the new semester, I've applied command-lines to the user accounts to gain access to manage the control system for password security, and verification with the proper expiration handling. When doing this project, it has defiantly strengthened my understating of the object oriented design by separating responsibility's throughout multiple classes, such as the class, "UserAccount" where the user is being used as a model and, "UserAccessManager", is the main system to manage the different system level operations.

One of the most key valuable aspects of this project i have observed and obtained was working with passwords security concepts. By using SHA-256 hashing through the provided Utilities.encryptedPassword method, i have learned firsthand how the system of the real world works by avoiding storing plaintext passwords and instead rely on one-way cryptography hashes. I have gained experience implanting an account from the lockout logic by tracking the failed login attempts and permanently locking accounts after repeated failures. This mirrors common security practices used in industry systems such as maybe company passwords or even school passwords.

When doing the initial testing of the code, I realized the significant and important role that JUnit tests play in ensuring code correctness and robustness. By using the provided GFA, UserAccessManagerGFATest, the public test suite, UserAccessManagerPublicTest, and the command-line integration tests, CommandLineIntegrationTests, I was able to validate both individual methods and the full expansion of the interactive workflow. Writing my own proper JUnit tests also improved my ability to think about edge cases, such as duplicate users, invalid inputs, and locked accounts. Throughout the entire project, I can say that I have improved my confidence in testing complex codes like this, where I needed to debug a Java application while also following detailed instructions and company-style security practices.

The image shows two screenshots of an IDE, likely IntelliJ, displaying Java code and test results.

Top Screenshot:

- Code Editor:** Shows `UserAccessManagerPublicTest.java`. The code includes a `@BeforeEach` `setUp()` method and a `@Test` `testAddUser()` method. The `testAddUser()` method calls `uam.addUser()` with username "charlie" and an encrypted password, then asserts that `uam.verifyAccess()` returns true for the same credentials.
- Run Window:** Shows the execution of `UserAccessManagerPublicTest`. It reports "6 tests passed" out of 6 tests total, taking 129 ms. The output shows the process finished with exit code 0.

Bottom Screenshot:

- Code Editor:** Shows `UserAccessManagerGFATest.java`. The code includes imports for JUnit 5, a `@Test` `testAddUserBasic()` method, and a `@Test` `testRemoveUserBasic()` method. The `testAddUserBasic()` method calls `uam.addUser()` with username "alice" and an encrypted password, then asserts that `assertDoesNotThrow()` returns true. The `testRemoveUserBasic()` method calls `uam.removeUser()` with username "alice" and asserts that `assertDoesNotThrow()` returns true.
- Run Window:** Shows the execution of `UserAccessManagerGFATest`. It reports "4 tests passed" out of 4 tests total, taking 201 ms. The output shows the process finished with exit code 0.

The image displays two screenshots of an IDE, likely IntelliJ IDEA, showing Java code and test results.

Top Screenshot: The main editor shows the `Main.java` file. The code implements a simple command-line interface for a user access manager. It uses a `Scanner` to read input and a `UserAccessManager` object to handle commands. The output shows the program running and responding to commands like `add david` and `verify david`.

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        UserAccessManager manager = new UserAccessManager();

        System.out.println("User access manager ready.");

        while (true) {
            System.out.print("User Access Manager> ");
            String input = scanner.nextLine().trim();

            if (input.isEmpty()) continue;

            String[] tokens = input.split(" ", 2);
            String command = tokens[0];
            String argument = tokens.length > 1 ? tokens[1].trim() : "";

            // ... (rest of the code) ...
        }
    }
}
```

The Run window shows the output of the program:

```
User access manager ready.
User Access Manager> add david
Password: mypassword
User 'david' added successfully.
User Access Manager> verify david
Password: wrongpass
Incorrect Password
User Access Manager> verify david
Password: mypassword
Access verified for 'david'.
User Access Manager>
```

Bottom Screenshot: The main editor shows the `UserAccountTest.java` file. The code contains three unit tests for the `UserAccount` class. The tests verify that accounts with the same username are equal, that accounts with different usernames are not equal, and that the `toString` method returns the correct username.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class UserAccountTest {

    @Test
    public void testEqualsSameUsername() {
        UserAccount a1 = new UserAccount("alice", "hash1");
        UserAccount a2 = new UserAccount("alice", "hash2");

        assertEquals(a1, a2, "Accounts with same username should be equal");
    }

    @Test
    public void testEqualsDifferentUsername() {
        UserAccount a1 = new UserAccount("alice", "hash");
        UserAccount a2 = new UserAccount("bob", "hash");

        assertNotEquals(a1, a2);
    }

    @Test
    public void testToStringReturnsUsername() {
        UserAccount user = new UserAccount("charlie", "hash");
        assertEquals("charlie", user.toString());
    }
}
```

The Run window shows the test results:

```
Run: UserAccountTest
41ms
6 tests passed, 6 tests total, 41ms
Process finished with exit code 0
```

The image shows two screenshots of an IDE interface, likely IntelliJ IDEA, demonstrating test execution results.

Top Screenshot: UserAccessManagerTest

- Project View:** Shows a project structure with a `test` directory containing `CommandLineIntegrationTests`, `UserAccessManagerPublicTest`, and `UserAccessManagerTest`.
- Code Editor:** Displays the `UserAccessManagerTest.java` file. It includes imports for JUnit 4 and contains two test methods:
 - `testAddUserSuccess()`: Tests adding a user with username "alice" and password "hash".
 - `testAddDuplicateUserThrowsException()`: Tests adding a user with username "bob" and password "hash", expecting an exception.
- Run Panel:** Shows the test run results for `UserAccessManagerTest`. The status is "11 tests passed" out of 11 tests total, taking 32 ms. The output shows the process finished with exit code 0.

Bottom Screenshot: CommandLineIntegrationTests

- Project View:** Shows the same project structure, with `CommandLineIntegrationTests` selected in the `test` directory.
- Code Editor:** Displays the `CommandLineIntegrationTests.java` file. It includes a helper method `runCliSession()` that simulates a CLI session by running a Java process with the given input.
- Run Panel:** Shows the test run results for `CommandLineIntegrationTests`. The status is "5 tests passed" out of 5 tests total, taking 706 ms. The output shows the process finished with exit code 0.